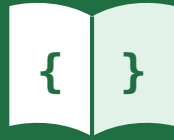


---

# Modül 18 — Spring Modulith

Ders Notları

Spring Boot Edu



2026-09-25

## İçindekiler

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| <b>1. Öğrenme Hedefleri</b>                                                                | <b>2</b>  |
| <b>2. Kavramlar</b>                                                                        | <b>2</b>  |
| 2.1 Modüler Monolit . . . . .                                                              | 2         |
| 2.2 Spring Modulith'te Modüller . . . . .                                                  | 2         |
| 2.3 Modüller Arası Event'ler . . . . .                                                     | 3         |
| <b>3. Adım Adım Örnekler</b>                                                               | <b>3</b>  |
| 3.1 Modül Yapısı ve <code>verify()</code> . . . . .                                        | 4         |
| 3.2 Bir Event Yayınlamak . . . . .                                                         | 5         |
| 3.3 Tepki Vermek: <code>@ApplicationModuleListener</code> . . . . .                        | 5         |
| 3.4 Event Publication Registry . . . . .                                                   | 6         |
| Kafka'ya Dışsallaştırma (Externalization) . . . . .                                        | 8         |
| 3.5 Modül Testleri: <code>@ApplicationModuleTest</code> ve <code>Scenario</code> . . . . . | 8         |
| 3.6 Dokümantasyon . . . . .                                                                | 9         |
| <b>4. Sık Yapılan Hatalar ve En İyi Pratikler</b>                                          | <b>10</b> |
| <b>5. Özet</b>                                                                             | <b>10</b> |
| <b>6. İleri Okuma</b>                                                                      | <b>11</b> |

## 1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Bir Spring Boot uygulamasını, iş modülü başına bir paketle modüler bir monolit olarak yapılandırmak
- Modül sınırlarını bir testte `ApplicationModules.verify()` ile kontrol etmek
- Modüllerin doğrudan çağrılar yerine application event'leri ve `@ApplicationModuleListener` ile konuşmasını sağlamak
- Event publication registry'nin ne sakladığını açıklamak ve listener'ı başarısız olan event'leri yeniden göndermek
- Seçili event'leri `@Externalized` ile Kafka'ya göndermek
- Her seferinde tek bir modülü `@ApplicationModuleTest` ve `Scenario` ile test etmek
- Mimari dokümantasyon üretmek (PlantUML ile C4 diyagramları, modül kanvasları)

**Ön koşullar:** Modül 06 (PostgreSQL, transaction'lar), Modül 11 (Kafka) · **Tahmini süre:** 4 saat · **Docker gerekir**

## 2. Kavramlar

### 2.1 Modüler Monolit

**Modüler monolit**, kodu net sınırları olan modüllere bölünmüş, tek başına deploy edilen bir uygulamadır. Bir monolitin basit işletimini (tek süreç, tek veritabanı, tek deployment) korur ve mikroservislerin yapısının büyük kısmını kazanır. İyi ayrılmış bir modül daha sonra kendi servisi olabilir. Her şeyin her şeyi çağırdığı bir monolitte bu çok daha zordur.

|                         | Klasik monolit     | Modüler monolit                          | Mikroservisler         |
|-------------------------|--------------------|------------------------------------------|------------------------|
| Deployment              | bir                | bir                                      | çok                    |
| Sınırlar                | gelenekle          | testlerle kontrol edilir                 | ağ tarafından zorlanır |
| Parçalar arası çağrılar | herhangi bir metot | modülün API'si veya event'ler            | HTTP, mesajlaşma       |
| Tutarlılık              | tek transaction    | modül başına bir transaction + event'ler | nihai (eventual)       |

### 2.2 Spring Modulith'te Modüller

Spring Modulith'in modülleri bulmak için hiçbir yapılandırmaya ihtiyacı yoktur:

- Ana uygulama paketinin her **doğrudan alt paketi** bir modüldür: order, inventory, catalog, notification.
- **Modülün kendi paketindeki** public tipler onun API'sidir.
- Bir modülün **alt paketleri** (order.internal), sınıfları Java için public olsa bile iç (internal) paketlerdir.

verify()'ın kontrol ettiği kurallar: Modüller arasında döngü yok, başka bir modülün iç tiplerine erişim yok ve bir modül yalnızca izin verdiği bağımlılıkları (allowedDependencies) kullanır.

## 2.3 Modüller Arası Event'ler

A modülü B modülünü doğrudan çağırdığında A, B'ye bağımlıdır ve B değiştiğinde değişmek zorundadır. Bir event ile A yalnızca olanı duyurur (OrderPlaced) ve istenen sayıda modül tepki verir. A bunların hiçbirini bilmez.

```
order —publishes→ OrderPlaced → inventory    (reserve stock)
                                ↳ notification (send confirmation)
                                ↳ Kafka topic   (other systems)
```

Listener, yayınlayan transaction'ın **commit'inden sonra**, kendi transaction'ında çalışır. Bu yüzden başarısız bir listener siparişi geri almaz. Ama o zaman stoğun hiç ayrılmadığını kim hatırlar? Bu, **event publication registry**'nin işidir.

## 3. Adım Adım Örnekler

Docker çalışırken uygulamayı başlatın. PostgreSQL ve Kafka kök compose.yaml'dan başlar:

```
./mvnw -pl modules/18-modulith/lesson spring-boot:run
```

Tur çıktısı:

```
=== 3.2-3.3 An order, and two modules that react to it ===
placed Order[id=1, customerId=tour, isbn=9780134685991, quantity=2, total=179.80]
inventory: 1000 → 998
mail: Thank you for order 1: 2 × 9780134685991, total 179.80
publication: inventory.StockReservations.on → COMPLETED
publication: notification.OrderConfirmation.on → COMPLETED
publication: EventExternalizerModuleListener.externalize → PROCESSING
=== 3.4 The warehouse is offline: the inventory listener fails ===
publication: inventory.StockReservations.on → FAILED
```

```
...
=== 3.4 Back online: resubmit the incomplete publications ===
publication: inventory.StockReservations.on → COMPLETED
...
```

### 3.1 Modül Yapısı ve verify()

Ders uygulamasında dört modül vardır:

| Modül        | API (modül paketi)                  | İç                                     | Bağımlı olduğu     |
|--------------|-------------------------------------|----------------------------------------|--------------------|
| catalog      | Book,<br>CatalogService             | catalog.internal.BookRepository        |                    |
| order        | Order, OrderPlaced,<br>OrderService | order.internal.OrderRepository         |                    |
| inventory    | Inventory,<br>Warehouse             | StockReservations<br>(package-private) | order (event tipi) |
| notification | Mailbox                             | OrderConfirmation<br>(package-private) | order (event tipi) |

Yapıyı normal bir birim testi kontrol eder. Ne Spring context'ine ne de Docker'a ihtiyaç duyar:

```
static final ApplicationModules MODULES =
    ↪ ApplicationModules.of(ModulithApplication.class);

@Test
void theModulesRespectTheirBoundaries() {
    MODULES.verify(); // no cycles, no access to another module's
    ↪ internal packages
}
```

inventory, order.internal.OrderRepository'yi kullansaydı test Module 'inventory' depends on non-exposed type ...OrderRepository within module 'order'! gibi bir mesajla başarısız olurdu.

Bir modül kendi bağımlılıklarını da kısıtlayabilir. Notification modülü yalnızca order'a bağımlı olabilir:

```
@ApplicationModule(allowedDependencies = "order")
```

**i Not**

Spring Modulith derlenmiş sınıfları okumak için ArchUnit kullanır. Modulith 2.1.1, Java 27 class dosyalarını okuyamayan ArchUnit 1.4.2'yi getirir ("No classes found"). Bu yüzden kursun build-parent'ı ArchUnit 1.5.0'ı yönetir.

## 3.2 Bir Event Yayınlamak

OrderPlaced yalnızca veri içeren bir record'dur: ID'ler, sayılar, entity yok. Listener'lar daha sonra ve başka transaction'larda çalışır ve registry event'i JSON olarak saklar:

```
@Externalized("bookstore.orders:#{customerId()}")    // also send to the Kafka topic,
↪ key = customer
public record OrderPlaced(long orderId, String customerId, String isbn, int quantity,
↪ BigDecimal total) {
}
```

Sipariş servisi onu Spring'in normal ApplicationEventPublisher'ı ile yayımlar:

```
@Transactional
public Order place(String customerId, String isbn, int quantity) {
    Book book = catalog.find(isbn).orElseThrow(() -> new UnknownBookException(isbn));
    BigDecimal total = book.price().multiply(BigDecimal.valueOf(quantity));
    Order order = repository.save(customerId, isbn, quantity, total);

    // stored in the registry in THIS transaction; delivered to the listeners after the
    ↪ commit
    events.publishEvent(new OrderPlaced(order.id(), customerId, isbn, quantity, total));
    return order;
}
```

Order modülü kimin dinlediğini bilmez. ModularityTest, onun yalnızca catalog'a bağımlı olduğunu kontrol eder.

## 3.3 Tepki Vermek: @ApplicationModuleListener

```
@ApplicationModuleListener    // = @TransactionalEventListener + @Async +
↪ @Transactional(REQUIRES_NEW)
void on(OrderPlaced order) {
    inventory.reserve(order.isbn(), order.quantity());
    warehouse.confirmReservation(order.isbn(), order.quantity());    // throws → rollback,
↪ publication FAILED
}
```

@ApplicationModuleListener üç anotasyonu birleştirir:

| Anotasyon                                  | Etki                                                            |
|--------------------------------------------|-----------------------------------------------------------------|
| @TransactionalEventListener                | yalnızca yayınlayan transaction commit edildikten sonra çalışır |
| @Async                                     | başka bir thread'de çalışır; sipariş isteği beklemez            |
| @Transactional(propagation = REQUIRES_NEW) | listener'ın kendi transaction'ı vardır                          |

Notification modülünün aynı event için ikinci bir listener'ı vardır (OrderConfirmation). Listener'ların hiçbirisi diğerini bilmez.

### 3.4 Event Publication Registry

spring-modulith-starter-jdbc ile her event, event\_publication tablosunda siparişle **aynı transaction'da listener başına bir satır** alır. Tablo Flyway tarafından oluşturulur (V2\_\_create\_event\_publication.sql, Modulith 2.1 PostgreSQL şemasından kopyalanmıştır).

| Durum       | Anlamı                                          |
|-------------|-------------------------------------------------|
| PUBLISHED   | sipariş commit edildi; listener henüz çalışmadı |
| PROCESSING  | listener çalışıyor                              |
| COMPLETED   | listener başarıyla bitti                        |
| FAILED      | listener bir exception fırlattı                 |
| RESUBMITTED | yayın listener'a yeniden verildi                |

Yapılandırma:

```
modulith:
  events:
    # the registry table is created by Flyway (V2); Modulith could also create it:
    # jdbc.schema-initialization.enabled: true
  externalization:
    enabled: true # send @Externalized events to Kafka (the default)
    completion-mode: update # completed publications stay in the table with a
    ↪ completion date
  kafka:
```

**producer:**

```
key-serializer: org.apache.kafka.common.serialization.StringSerializer
# no JSON value serializer: Modulith turns the event into JSON itself
# (spring.modulith.events.kafka.enable-json, default true) – a JsonSerializer would
↪ encode it twice
```

Test depoyu kapatır, böylece inventory listener'ı başarısız olur. Sipariş commit edilmiş kalır ve registry başarısız yayını hatırlar. Depo geri geldikten sonra yayın yeniden gönderilir:

```
@Test
void aFailedListenerIsDeliveredAgainFromTheRegistry() {
    int before = inventory.available(JAVA_PUZZLERS);
    warehouse.goOffline(); // the inventory listener
    ↪ will throw

    Order order = orders.place("c-3", JAVA_PUZZLERS, 1); // the order itself is
    ↪ committed

    await().atMost(Duration.ofSeconds(10))
        .untilAsserted(() ->
            ↪ assertThat(inventoryPublicationStatus(order.id())).isEqualTo("FAILED"));
    assertThat(inventory.available(JAVA_PUZZLERS)).isEqualTo(before);

    warehouse.goOnline();
    incompletePublications.resubmitIncompletePublications(
        publication -> publication.getEvent() instanceof OrderPlaced placed &&
    ↪ placed.orderId() == order.id());

    await().atMost(Duration.ofSeconds(10))
        .untilAsserted(() -> assertT-
            ↪ hat(inventoryPublicationStatus(order.id())).isEqualTo("COMPLETED"));
    assertThat(inventory.available(JAVA_PUZZLERS)).isEqualTo(before - 1);
}
```

Production'da genellikle zamanlanmış bir iş `resubmitIncompletePublicationsOlderThan(Duration)`'ı çağırır ve `CompletedEventPublications.deletePublicationsOlderThan(...)` tabloyu küçük tutar. `spring.modulith.events.republish-outstanding-events-on-restart=true` ile bitmemiş yayınlar yeniden başlatmadan sonra da tekrar iletilir.

**! Önemli**

Registry tam olarak bir kez değil, **en az bir kez** garanti eder. Bir çöküşten sonra bir listener aynı event için iki kez çalışabilir. Listener'lar idempotent olmalı veya tekrarları tespit etmelidir (modül 11'deki outbox'ın tüketici tarafıyla karşılaştırın).

## Kafka'ya Dışsallaştırma (Externalization)

Event üzerindeki `@Externalized("bookstore.orders::#{customerId()}")`, Modulith'in onu anahtar olarak müşteri ID'siyle `bookstore.orders` Kafka topic'ine göndermesini sağlar. Bu da registry satırı olan bir listener'dır (`EventExternalizerModuleListener.externalize`): Registry bir **outbox** olarak çalışır. Outbox tablosunu ve relay'i modül 11'deki gibi kendiniz yazmazsınız. `ExternalizationTest` topic'i düz bir Kafka consumer'ı ile okur.

### ! Uyarı

Modulith event'i JSON'a kendisi çevirir (`spring.modulith.events.kafka.enable-json`, varsayılan `true`). Ayrıca bir JSON value-serializer yapılandırırsanız event iki kez kodlanır ve base64 bir string olarak gelir.

## 3.5 Modül Testleri: `@ApplicationModuleTest` ve Scenario

`@ApplicationModuleTest`, context'i yalnızca **tek** bir modülün (testin paketindeki modülün) bean'leriyle başlatır. Diğer modüller orada değildir ve bean'leri mock'lanabilir:

```
@ApplicationModuleTest                                // STANDALONE: the beans of this module
↳ only
@Import(TestcontainersConfiguration.class)
@TestPropertySource(properties = "bookstore.tour.enabled=false") // the tour needs all
↳ modules
class OrderModuleTest {

    @MockitoBean
    CatalogService catalog;                            // another module: replaced by a mock

    @Autowired
    OrderService orders;

    @Test
    void placingAnOrderPublishesOrderPlaced(Scenario scenario) {
        given(catalog.find("9780134685991"))
            .willReturn(Optional.of(new Book("9780134685991", "Effective Java", new
                ↳ BigDecimal("89.90"))));

        scenario.stimulate(() -> orders.place("c-1", "9780134685991", 2))
            .waitForEventOfType(OrderPlaced.class)
            .matching(event -> event.customerId().equals("c-1"))
            .toArriveAndVerify(event ->
                ↳ assertThat(event.total()).isEqualByComparingTo("179.80"));
    }
}
```

| Bootstrap modu          | Başlatılan modüller                             |
|-------------------------|-------------------------------------------------|
| STANDALONE (varsayılan) | yalnızca test edilen modül                      |
| DIRECT_DEPENDENCIES     | artı doğrudan bağımlı olduğu modüller           |
| ALL_DEPENDENCIES        | artı geçişli olarak bağımlı olduğu tüm modüller |

Scenario bir testi **uyarı** → **beklenen sonuç** olarak tarif eder. Uyarı bir metot çağrısı (stimulate) veya bir event (publish) olabilir. Sonuç bir event (awaitForEventOfType) veya bir durum değişikliği (awaitForStateChange) olabilir. Scenario asenkron listener'ları Awaitility ile bekler:

```
@Test
void anOrderReservesStock(Scenario scenario) {
    int before = inventory.available(ENGINE_IN_ACTION);

    scenario.publish(new OrderPlaced(1000, "c-2", ENGINE_IN_ACTION, 3, new
    ↪ BigDecimal("285.00")))
        .awaitForStateChange(() -> inventory.available(ENGINE_IN_ACTION), available
        ↪ -> available != before)
        .andVerify(available -> assertThat(available).isEqualTo(before - 3));
}
```

#### ★ İpucu

@ApplicationModuleTest uygulamanın kök paketini de tarar. Oradaki LessonTour tüm modüllere ihtiyaç duyar, bu yüzden modül testleri onu @TestPropertySource(properties = "bookstore.tour.enabled=false") ile kapatır.

## 3.6 Dokümantasyon

```
@Test
void writeDocumentation() {
    new Documenter(MODULES).writeDocumentation();    // PlantUML (C4) diagrams + module
    ↪ canvases

    assertThat(Path.of("target/spring-modulith-docs/components.puml")).exists();
    assertThat(Path.of("target/spring-modulith-docs/module-order.puml")).exists();
}
```

./mvnw -pl modules/18-modulith/lesson test -Dtest=ModularityTest sonrasında lesson/target/spring-modulith-docs klasörü şunları içerir:

- `components.puml`: Tüm modüllerin ve bağımlılıklarının bir C4 bileşen diyagramı
- `module-<name>.puml`: Modül başına bir diyagram
- `module-<name>.adoc`: Bir **modül kanvası** (API tipleri, Spring bean'leri, yayınlanan ve tüketilen event'ler, property'ler)
- `all-docs.adoc`: Hepsi tek bir dokümanda

.puml dosyalarını bir PlantUML eklentisiyle (IntelliJ IDEA, VS Code) veya plantuml.com'da açın. Bir test tarafından üretildikleri için dokümantasyon koddan uzaklaşamaz.

## 4. Sık Yapılan Hatalar ve En İyi Pratikler

### !! Dikkat

Event'lere entity'ler veya değiştirilebilir başka nesneler koymayın. Listener daha sonra, başka bir transaction'da, belki bir yeniden başlatmadan sonra registry'deki JSON'dan çalışır. ID'leri ve listener'ların ihtiyaç duyduğu değerleri koyun.

- **Yapın:** Modül API'sini bilinçli tasarlayın: Modül paketinde az sayıda public tip, geri kalan her şey alt paketlerde.
- **Yapmayın:** İç bir paketteki bir sınıfı public yapıp başka bir modülden "sadece bu seferlik" kullanmayın. `verify()` başarısız olur ve amaç da budur.
- **Yapın:** `verify()`'ı (veya `detectViolations()`'ı) her build'de çalıştırın. `ApplicationModules.of(...)` JVM başına ön belleğe alınır ve bir `@ApplicationModuleTest` onu bir kez doğruladıktan sonra `verify()` tekrar kontrol etmez. `detectViolations().throwIfPresent()` her zaman kontrol eder.
- **Yapmayın:** Her metot çağrısını bir event'le değiştirmeyin. Sorgular ("bu kitap ne kadar?") diğer modülün API'sine çağrı olarak kalır. Event'ler "bu oldu" içindir.
- **Yapın:** Listener'ları idempotent yapın ve FAILED yayınları izleyin.
- **Yapmayın:** Tamamlanmış yayınları temizlemeyi (`CompletedEventPublications`) unutmayın, yoksa tablo sonsuza dek büyür.

## 5. Özet

- Paketler modüllerdir: Modül paketi API'dir ve alt paketler iç paketlerdir. `ApplicationModules.verify()` bunu bir birim testinde kontrol eder.
- Modüller event'lerle iletişim kurar. `@ApplicationModuleListener` `commit`'ten sonra, asenkron olarak, kendi transaction'ında çalışır.
- Event publication registry, event ve listener başına bir satırı aynı transaction'da saklar. Başarısız yayınlar yeniden gönderilebilir: en az bir kez iletim.
- `@Externalized` bir event'i Kafka'ya gönderir ve registry outbox görevi görür.
- `@ApplicationModuleTest` tek bir modülü başlatır ve Scenario event'leri ve durum değişikliklerini test eder. Documenter C4 diyagramları ve modül kanvasları üretir.

Sıradaki adım: [Ödevler](#)

## 6. İleri Okuma

- [Spring Modulith Reference](#) · [Fundamentals](#) · [Verification](#)
- [Working with Application Events](#)
- [Integration Testing Application Modules](#)
- [Documenting Application Modules](#)
- [Martin Fowler — Monolith First](#)